МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Выполнил: С. М. Закатин
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы:

Приобретение практических навыков управления потоками в операционной системе и обеспечения синхронизации между потоками.

Задание:

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы Unix. Максимальное количество одновременно работающих потоков должно задаваться ключом запуска программы. Необходимо продемонстрировать количество используемых потоков стандартными средствами операционной системы, а также исследовать зависимость ускорения и эффективности алгоритма от входных данных и количества потоков.

Вариант: 13

Наложить K раз фильтр, использующий матрицу свёртки, на матрицу, состоящую из вещественных чисел. Размер окна задаётся пользователем.

Метод решения

Исходная матрица, квадратное ядро свёртки и количество повторений K считываются из файла или стандартного ввода. Для элемента результирующей матрицы используется двумерная дискретная свёртка

$$D(i,j) = \sum_{u=0}^{W-1} \sum_{v=0}^{W-1} S(i+u-r, j+v-r)\, H(W-1-u, W-1-v),$$

где S — исходная матрица, D — результат, H — ядро размером $W \times W$, $r = \lfloor W/2 \rfloor$. Индексы ядра отражаются по обеим координатам, поэтому реализована именно свёртка, а не корреляция. Элементы за границами исходной матрицы считаются равными нулю. Физически расширенная матрица не создаётся: недопустимые индексы пропускаются.

Матрица хранится в одном непрерывном блоке памяти. Структура `matrix_t` содержит указатель на данные, число строк и столбцов и шаг между соседними строками `stride`. Адрес элемента вычисляется как `data + row * stride + col`. Сейчас шаг равен числу столбцов, однако отдельное поле позволяет в дальнейшем добавить выравнивание строк без изменения алгоритмов.

Свёртка не выполняется на месте. При нескольких повторениях используются результирующая и временная матрицы. После каждого прохода они меняются ролями. Первый выходной буфер выбирается по чётности K, благодаря чему после последней итерации результат всегда находится в переданной матрице назначения:

$K = 2$: `source -> temporary -> destination`,

$K = 3$: `source -> destination -> temporary -> destination`.

При $K = 0$ исходная матрица просто копируется в результат.

Для повышения эффективности порядок циклов изменён относительно прямой реализации `row-column-kernel row-kernel column`. Для фиксированных строки результата и элемента ядра обрабатывается непрерывный диапазон столбцов. Внутренний цикл имеет

вид `destination[i] += source[i] * weight`, последовательно обращается к памяти и в сборке `Release` векторизуется компилятором. Квалификатор `restrict` сообщает компилятору, что обрабатываемые фрагменты входного и выходного буферов не перекрываются.

Параллельная обработка выполняется постоянным пулом POSIX-потоков. Матрица делится на непересекающиеся диапазоны строк. Каждая задача содержит ссылки на исходную матрицу, результат и ядро, а также полуинтервал строк `[row_begin, row_end)`. Исходная матрица и ядро доступны потокам только для чтения, а каждый поток записывает собственные строки результата. Поэтому мьютекс во время вычисления свёртки не требуется.

Количество задач адаптируется к размеру входа. Оцениваемое число операций для одной строки равно `cols * kernel_rows * kernel_cols`. Задача должна содержать примерно не менее 65536 операций, чтобы затраты на очередь, пробуждение worker и барьер не превышали полезную работу. Итоговое число задач не превосходит количества рабочих потоков и строк матрицы. Если полезна только одна задача, она выполняется непосредственно главным потоком без обращения к очереди.

После постановки задач каждой итерации главный поток ожидает одновременно пустой очереди и нулевого числа активных workers. Этот барьер обязателен: входной и выходной буферы разрешается менять местами только после завершения всех записей текущей итерации.

Описание программы

Основные файлы проекта:

- `main.c` — разбор аргументов, открытие входного файла, создание матриц и пула, измерение времени и вывод результата;
- `input.c`, `input.h` — проверяемое чтение размеров и вещественных элементов;
- `matrix.c`, `matrix.h` — хранение матриц, проверка переполнения размеров, копирование, клонирование и освобождение;
- `convolution.c`, `convolution.h` — последовательная и параллельная свёртка, повторные проходы и адаптивное разбиение на задачи;
- `common/os/thread_pool.c` — пул потоков и кольцевая очередь;
- `common/os/posix/mutex.h` — обёртка над POSIX-мьютексом и scoped lock;
- `common/raii.h` — RAII-подобные типы, операции создания, перемещения и клонирования ресурсов;
- `CMakeLists.txt` — конфигурация сборки и подключение библиотеки потоков.

Кольцевая очередь пула описывается индексами `queue_head` и `queue_tail`, числом ожидающих задач `queued_count` и числом выполняемых задач `active_count`. Все эти поля защищены одним мьютексом. Используются три условные переменные:

- `task_available` — ожидание worker до появления задачи;
- `queue_space_available` — ожидание producer до освобождения места в очереди;
- `idle` — ожидание главным потоком полного завершения работы.

Ожидания оформлены циклами `while`, поскольку POSIX допускает ложные пробуждения, а после пробуждения другой worker может первым изменить состояние очереди. Worker извлекает задачу и увеличивает `active_count` под мьютексом, но выполняет функцию

задачи после снятия блокировки. После завершения он снова захватывает мьютекс, уменьшает счётчик и сигнализирует idle, если очередь пуста и активных задач больше нет.

Задачи одной итерации передаются в очередь одним пакетом. Это сокращает количество захватов мьютекса по сравнению с добавлением каждой задачи отдельно. Если пакет не помещается, он передаётся частями, между которыми mutex освобождается, чтобы workers могли освободить место.

Для автоматического управления ресурсами используется атрибут GCC/Clang cleanup. RAII-переменные матриц, пула и scoped lock вызывают деструктор при любом выходе из области видимости. Операция move переносит указатель на ресурс и переводит источник в каноническое невалидное состояние, предотвращая двойное освобождение. Сам pthread_mutex_t не перемещается побитовым копированием; структура mutex_t дополнительно хранит флаг успешной инициализации.

Числа сначала читаются как текстовые токены и преобразуются функциями strtoumax и strtod. Отклоняются отрицательные и дробные размеры, переполнение, незавершённые данные, NaN и бесконечность. Матрицы сначала заполняются во временных RAII-объектах и только после полного успеха перемещаются в результат, поэтому ошибка не оставляет частично заполненных ресурсов.

В программе непосредственно используются следующие функции POSIX:

- pthread_create и pthread_join — создание и ожидание рабочих потоков;
- pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock и pthread_mutex_destroy — защита состояния пула;
- pthread_cond_init, pthread_cond_wait, pthread_cond_signal, pthread_cond_broadcast и pthread_cond_destroy — ожидание изменений состояния;
- clock_gettime с часами CLOCK_MONOTONIC — измерение времени без влияния изменения системных часов.

При частичной ошибке инициализации пула уже созданные потоки останавливаются и объединяются, после чего условные переменные, мьютекс и память освобождаются в обратном порядке. При штатном завершении RAII-деструктор устанавливает флаг stopping, пробуждает workers, вызывает pthread_join и освобождает все внутренние ресурсы.

Результаты

Проект был собран GCC 15.2.1 при помощи CMake в конфигурации Release с флагами -O3 -DNDEBUG. Измерения выполнены на процессоре Intel Core i5-13420H, содержащем четыре производительных ядра с Hyper-Threading и четыре энергоэффективных ядра, всего 12 логических CPU. Операционная система — Fedora Linux, потоковая реализация — POSIX Threads.

Для тестирования использован файл large_512x512.in: матрица 512×512, нормированное ядро размытия 9×9 и $K = 10$ повторений. Без учёта граничных элементов это соответствует примерно $512 \cdot 512 \cdot 9^2 \cdot 10 = 212\,336\,640$ операциям умножения с накоплением. Вывод результата отключался ключом -no-output.

Для каждого количества потоков выполнено три прогревочных и 30 измеряемых запусков. В таблице приведена медиана внутреннего времени, измеренного функцией clock_gettime. Чтение файла, создание пула и печать результата находятся вне измеряемого

интервала. Ускорение и эффективность вычисляются по формулам

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p} \cdot 100\%.$$

Потоки p	Время T_p, мс	Ускорение S_p	Эффективность E_p
1	107,346	1,000	100,0%
2	59,803	1,795	89,8%
4	35,265	3,044	76,1%
8	28,556	3,759	47,0%
12	26,038	4,123	34,4%
20	24,786	4,331	21,7%

Таблица 1 — Зависимость времени, ускорения и эффективности от числа потоков

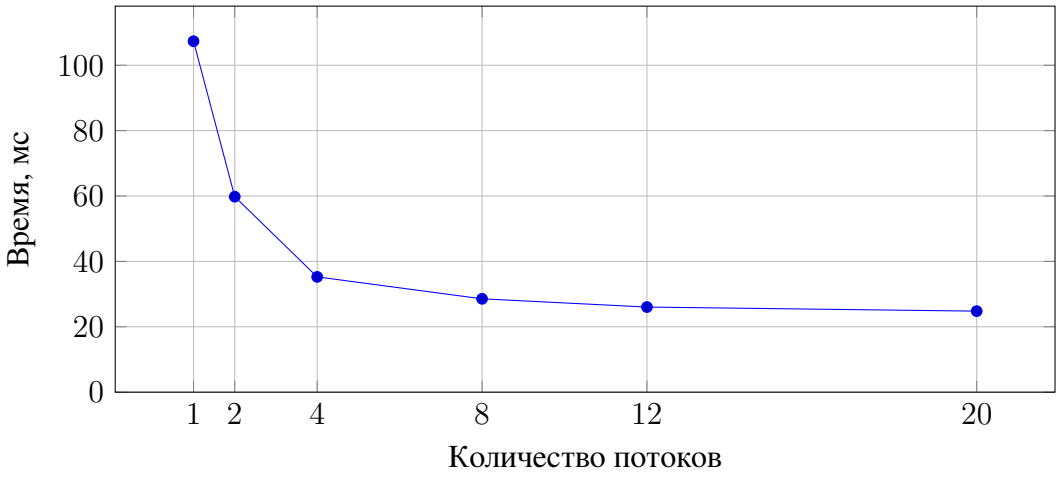


Рис. 1: Зависимость времени выполнения от количества потоков

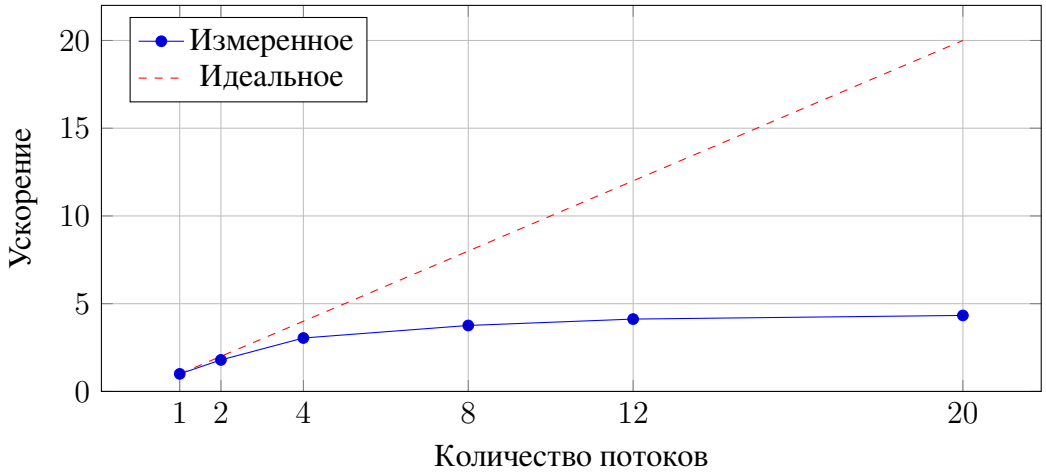


Рис. 2: Измеренное и идеальное ускорение

Переход от одного потока к двум даёт ускорение 1,795 при эффективности 89,8%. На четырёх потоках ускорение достигает 3,044. Дальнейшее увеличение числа workers продолжает уменьшать время, однако прирост становится небольшим: между 12 и 20 потоками

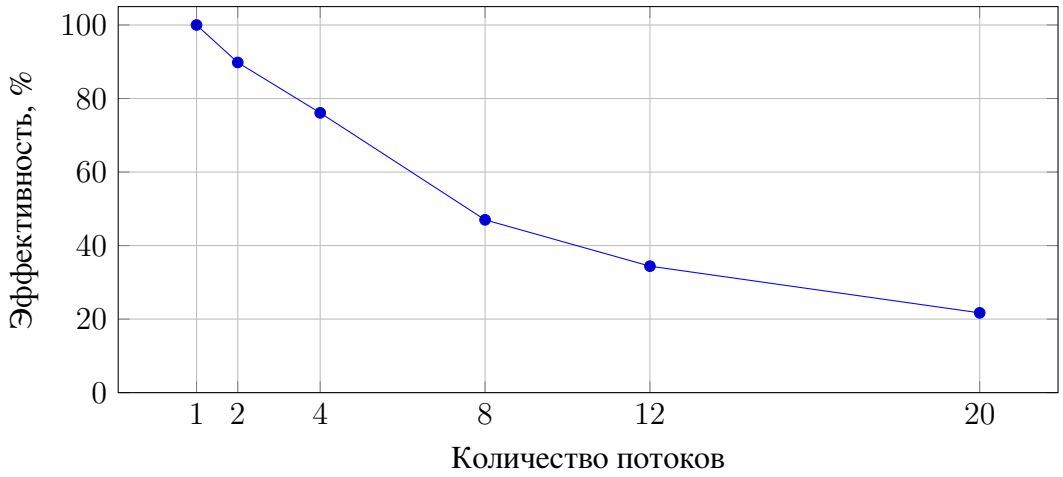


Рис. 3: Зависимость эффективности от количества потоков

время сокращается только с 26,038 до 24,786 мс. При 20 workers ускорение равно 4,331, но эффективность снижается до 21,7%.

Причина нелинейного масштабирования состоит в ограниченном числе физических ядер, общей пропускной способности памяти и барьере после каждого из десяти проходов. После четырёх потоков в работу вовлекаются энергоэффективные ядра и логические потоки P-ядер с меньшей производительностью на один поток. При 20 workers число рабочих потоков превышает 12 логических CPU, поэтому возникает конкуренция за процессорное время. Дополнительные затраты создают mutex очереди, condition variables и изменение общих счётчиков пула.

Для определения наиболее затратных функций дополнительно выполнено профилирование утилитой `perf c` частотой 997 Гц. Чтобы получить достаточное число сэмплов, количество повторений было увеличено до $K = 500$. Стек вызовов записывался с указателями кадров, после чего данные P- и E-ядер были объединены и представлены в виде FlameGraph на рисунке 4. Ширина прямоугольника на FlameGraph соответствует доле собранных сэмплов, а не длительности отдельного вызова.

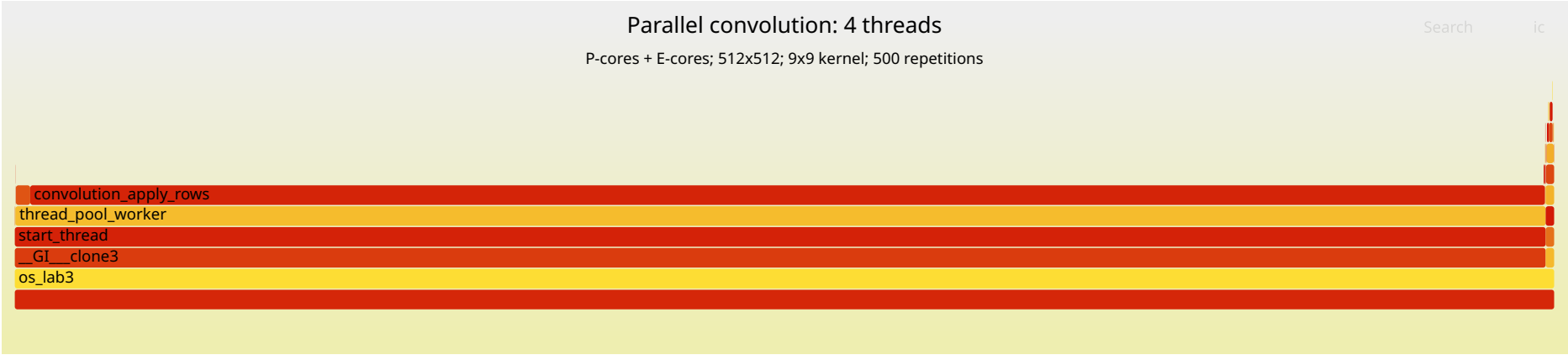


Рис. 4: FlameGraph параллельной свёртки при четырёх рабочих потоках

На графике `thread_pool_worker` занимает 99,41% сэмплов, поскольку является родительской функцией для вычислительной задачи. Вложенная функция `convolution_apply_rows` занимает 98,40%, тогда как чтение входа — около 0,53%. Следовательно, на большом входе основное время занимает полезное вычисление свёртки, а не управление очередью. Ширина worker не означает, что 99,41% времени расходуется на mutex: в неё включено время всех вызванных функций. На малых входах соотношение меняется, поэтому реализован последовательный путь и адаптивное число задач.

Функциональные тесты проверяют единичное ядро, нулевое число повторений, матрицу 1×1, обработку границ с нулевым дополнением и отражение несимметричного ядра. Отдельные некорректные входы содержат нулевые и отрицательные размеры, чётный размер окна, дробные размеры, нечисловые и неограниченные значения, а также оборванные данные. Все корректные тесты прошли с одним и четырьмя потоками, все некорректные входы завершились с ненулевым кодом возврата. Сборка с AddressSanitizer и UndefinedBehaviorSanitizer также завершила тесты без обнаруженных ошибок.

Фактическое число потоков проверялось командой:

```
./build-release/os_lab3 -t 4 -i /tmp/large_512x512_k500.in -q &
pid=$!
ps -L -p "$pid" -o pid,tid,psr,stat,comm
```

Утилита `ps` показала пять LWP: главный поток и четыре worker-потока. Главный поток после отправки задач ожидает барьер, поэтому свёртку параллельно вычисляют не более четырёх workers, как задано ключом `-t 4`.

Выводы

В ходе лабораторной работы реализована многопоточная программа для повторного наложения произвольного квадратного ядра свёртки на вещественную матрицу. Работа разделяется по диапазонам строк, поэтому вычислительные потоки совместно читают исходные данные, но записывают непересекающиеся области результата. Такой способ декомпозиции устраняет необходимость блокировать матрицу во внутреннем цикле и оставляет синхронизацию только в инфраструктуре пула.

Практически важным оказалось разделение состояния задачи на ожидающее и активное. Одной проверки пустоты очереди недостаточно: worker может уже извлечь последнюю задачу, но ещё не закончить её. Совместная проверка `queued_count == 0` и `active_count == 0` формирует корректный барьер между повторениями фильтра. Циклы вокруг `pthread_cond_wait` защищают программу от ложных пробуждений и конкуренции нескольких workers.

Для повторных проходов применена схема ping-pong из двух выходных буферов. Выбор первого буфера по чётности K гарантирует размещение окончательного результата в матрице назначения без дополнительного копирования. Изменение порядка циклов сделало внутреннюю обработку последовательной по памяти и позволило компилятору использовать SIMD. Пакетная постановка задач уменьшила число захватов мьютекса, а адаптивная гранулярность исключает затраты пула на слишком маленьких входах.

Эксперимент показал ускорение 3,044 на четырёх потоках и 4,331 на двадцати. При этом эффективность снизилась с 89,8% для двух потоков до 21,7% для двадцати. Результат объясняется законом Амдала, ограниченным количеством физических и логических ядер, неоднородностью P- и E-ядер процессора, общей пропускной способностью памяти и необходимостью синхронизации после каждого прохода. Поэтому увеличение числа

потоков выше числа доступных аппаратных потоков даёт лишь небольшой прирост и не является эффективным использованием вычислительных ресурсов.

RAII-подобные типы на основе расширений GCC и Clang обеспечили автоматическое освобождение матриц, пула и блокировок при обычном и досрочном выходе из функции. Вместе с явной проверкой результатов инициализации, тестами и санитайзерами это позволило безопасно обрабатывать частичную инициализацию и ошибочные входные данные.

Исходная программа

```c
#include <errno.h>
#include <inttypes.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#include "convolution.h"
#include "input.h"

typedef struct {
    size_t thread_count;
    const char *input_path;
    bool has_thread_count;
    bool suppress_output;
} program_options_t;

typedef enum {
    OPTIONS_OK = 0,
    OPTIONS_HELP,
    OPTIONS_ERROR
} options_result_t;

static void print_usage(FILE *stream, const char *program_name) {
    fprintf(stream,
        "Usage: %s --threads COUNT [--input FILE]\n"
        " %s -t COUNT [-i FILE]\n"
        "\n"
        "Options:\n"
        " -t, --threads COUNT maximum number of worker threads\n"
        " -i, --input FILE read input from FILE instead of stdin\n"
        " -q, --no-output do not print the resulting matrix\n"
        " -h, --help show this help\n",
        program_name, program_name);
}

static bool parse_positive_size(const char *text, size_t *value) {
    if (text == NULL || value == NULL || text[0] == '\0' || text[0] == '-') {
        return false;
    }

    char *end = NULL;
    errno = 0;
    const uintmax_t parsed = strtoumax(text, &end, 10);
    if (end == text || *end != '\0' || errno == ERANGE
        || parsed == 0 || parsed > SIZE_MAX) {
        return false;
    }

    *value = (size_t) parsed;
    return true;
}

static options_result_t parse_options(
    int argc,
```

```
    char **argv,
    program_options_t *options
) {
    *options = (program_options_t) {0};

    for (int index = 1; index < argc; ++index) {
        const char *argument = argv[index];

        if (strcmp(argument, "-h") == 0 || strcmp(argument, "--help") == 0) {
            return OPTIONS_HELP;
        }

        if (strcmp(argument, "-t") == 0 || strcmp(argument, "--threads") == 0) {
            if (options->has_thread_count) {
                fprintf(stderr, "Thread count was specified more than once\n");
                return OPTIONS_ERROR;
            }
            if (++index == argc) {
                fprintf(stderr, "Missing value after %s\n", argument);
                return OPTIONS_ERROR;
            }
            if (!parse_positive_size(argv[index], &options->thread_count)) {
                fprintf(stderr, "Invalid thread count: %s\n", argv[index]);
                return OPTIONS_ERROR;
            }
            options->has_thread_count = true;
            continue;
        }

        if (strcmp(argument, "-i") == 0 || strcmp(argument, "--input") == 0) {
            if (options->input_path != NULL) {
                fprintf(stderr, "Input file was specified more than once\n");
                return OPTIONS_ERROR;
            }
            if (++index == argc) {
                fprintf(stderr, "Missing value after %s\n", argument);
                return OPTIONS_ERROR;
            }
            options->input_path = argv[index];
            continue;
        }

        if (strcmp(argument, "-q") == 0 || strcmp(argument, "--no-output") == 0) {
            options->suppress_output = true;
            continue;
        }

        fprintf(stderr, "Unknown argument: %s\n", argument);
        return OPTIONS_ERROR;
    }

    if (!options->has_thread_count) {
        fprintf(stderr, "The --threads option is required\n");
        return OPTIONS_ERROR;
    }
    return OPTIONS_OK;
}

```

```c
static bool matrix_print(FILE *stream, const matrix_t *matrix) {
    if (!matrix_is_valid(matrix)) return false;

    if (fprintf(stream, "%zu %zu\n", matrix->rows, matrix->cols) < 0) {
        return false;
    }

    for (size_t row = 0; row < matrix->rows; ++row) {
        const double *values = matrix_const_row(matrix, row);
        for (size_t col = 0; col < matrix->cols; ++col) {
            const char *separator = col + 1 == matrix->cols ? "\n" : " ";
            if (fprintf(stream, "%.17g%s", values[col], separator) < 0) {
                return false;
            }
        }
    }
    return true;
}

static const double NANO_TO_SEC = 1 / 1e9;

static double elapsed_seconds(struct timespec begin, struct timespec end) {
    const time_t seconds = end.tv_sec - begin.tv_sec;
    const long nanoseconds = end.tv_nsec - begin.tv_nsec;
    return (double) seconds + (double) nanoseconds * NANO_TO_SEC;
}

int main(int argc, char **argv) {
    program_options_t options;
    const options_result_t options_result = parse_options(argc, argv, &options);
    if (options_result == OPTIONS_HELP) {
        print_usage(stdout, argv[0]);
        return EXIT_SUCCESS;
    }
    if (options_result == OPTIONS_ERROR) {
        print_usage(stderr, argv[0]);
        return EXIT_FAILURE;
    }

    FILE *input_stream = stdin;
    if (options.input_path != NULL) {
        input_stream = fopen(options.input_path, "r");
        if (input_stream == NULL) {
            fprintf(stderr, "Cannot open input file '%s': %s\n",
                options.input_path, strerror(errno));
            return EXIT_FAILURE;
        }
    }

    raii_stack(matrix) source = create_stack(matrix);
    raii_stack(matrix) kernel = create_stack(matrix);
    size_t repeat_count;
    const input_error_t input_error = input_read(input_stream, &source, &kernel, &
        repeat_count);

    if (input_stream != stdin && fclose(input_stream) != 0) {
        fprintf(stderr, "Cannot close input file '%s': %s\n",
            options.input_path, strerror(errno));
```

```
172         return EXIT_FAILURE;
173     }
174     if (input_error != INPUT_OK) {
175         fprintf(stderr, "Input error: %s\n", input_error_string(input_error));
176         return EXIT_FAILURE;
177     }
178 
179     matrix_error_t matrix_error;
180     raii_stack(matrix) result = new_stack(matrix, &matrix_error, source.rows, source.
        cols);
181     if (matrix_error != MATRIX_OK) {
182         fprintf(stderr, "Cannot allocate result matrix (error %d)\n", matrix_error);
183         return EXIT_FAILURE;
184     }
185 
186     thread_pool_error_t pool_error;
187     raii_stack(thread_pool) pool = new_stack(
188         thread_pool, &pool_error, options.thread_count, options.thread_count);
189     if (pool_error != THREAD_POOL_OK) {
190         fprintf(stderr, "Thread pool error: %s\n",
191             thread_pool_error_string(pool_error));
192         return EXIT_FAILURE;
193     }
194 
195     struct timespec begin;
196     struct timespec end;
197     if (clock_gettime(CLOCK_MONOTONIC, &begin) != 0) {
198         perror("clock_gettime");
199         return EXIT_FAILURE;
200     }
201 
202     const convolution_error_t convolution_error =
203         convolution_apply_repeated_parallel(
204             &result, &source, &kernel, repeat_count, &pool);
205 
206     if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
207         perror("clock_gettime");
208         return EXIT_FAILURE;
209     }
210     if (convolution_error != CONVOLUTION_OK) {
211         fprintf(stderr, "Convolution error: %s\n",
212             convolution_error_string(convolution_error));
213         return EXIT_FAILURE;
214     }
215 
216     if (!options.suppress_output && !matrix_print(stdout, &result)) {
217         fprintf(stderr, "Output error\n");
218         return EXIT_FAILURE;
219     }
220 
221     fprintf(stderr,
222         "threads=%zu repetitions=%zu elapsed_seconds=%.9f\n",
223         thread_pool_thread_count(&pool), repeat_count,
224         elapsed_seconds(begin, end));
225     return EXIT_SUCCESS;
```

```
}
```

Листинг 1: Главная функция программы

```
#ifndef INPUT_H
#define INPUT_H

#include <stddef.h>
#include <stdio.h>

#include "matrix.h"

typedef enum {
    INPUT_OK = 0,
    INPUT_INVALID_ARGUMENT,
    INPUT_DESTINATION_NOT_EMPTY,
    INPUT_UNEXPECTED_END,
    INPUT_INVALID_NUMBER,
    INPUT_INVALID_WINDOW_SIZE,
    INPUT_MATRIX_SIZE_ERROR,
    INPUT_ALLOCATION_FAILED,
    INPUT_IO_ERROR
} input_error_t;

/*
Expected format (whitespace is insignificant):

     rows cols
     window_size repeat_count
     <rows * cols matrix elements>
     <window_size * window_size kernel elements>

source and kernel must be distinct matrices in their reset state. On failure they
    remain in that state, and repeat_count is not modified.
 */
input_error_t input_read(
    FILE *stream,
    matrix_t *source,
    matrix_t *kernel,
    size_t *repeat_count);

const char *input_error_string(input_error_t error);

#endif
```

Листинг 2: Интерфейс чтения входных данных

```
#include "input.h"

#include <errno.h>
#include <inttypes.h>
#include <math.h>
#include <stdlib.h>

typedef enum {
    TOKEN_OK = 0,
    TOKEN_END,
```

```c
    TOKEN_INVALID,
    TOKEN_IO_ERROR
} token_error_t;

static token_error_t read_token(FILE *stream, char token[128]) {
    int result = fscanf(stream, "%127s", token);
    if (result == 1) return TOKEN_OK;
    if (ferror(stream)) return TOKEN_IO_ERROR;
    return TOKEN_END;
}

static token_error_t read_size(FILE *stream, size_t *value) {
    char token[128];
    token_error_t error = read_token(stream, token);
    if (error != TOKEN_OK) return error;

    if (token[0] == '-') return TOKEN_INVALID;

    char *end = NULL;
    errno = 0;
    uintmax_t parsed = strtoumax(token, &end, 10);
    if (end == token || *end != '\0' || errno == ERANGE || parsed > SIZE_MAX) {
        return TOKEN_INVALID;
    }

    *value = (size_t) parsed;
    return TOKEN_OK;
}

static token_error_t read_double(FILE *stream, double *value) {
    char token[128];
    token_error_t error = read_token(stream, token);
    if (error != TOKEN_OK) return error;

    char *end = NULL;
    errno = 0;
    double parsed = strtod(token, &end);
    if (end == token || *end != '\0' || errno == ERANGE || !isfinite(parsed)) {
        return TOKEN_INVALID;
    }

    *value = parsed;
    return TOKEN_OK;
}

static input_error_t token_to_input_error(token_error_t error) {
    switch (error) {
        case TOKEN_OK:
            return INPUT_OK;
        case TOKEN_END:
            return INPUT_UNEXPECTED_END;
        case TOKEN_INVALID:
            return INPUT_INVALID_NUMBER;
        case TOKEN_IO_ERROR:
            return INPUT_IO_ERROR;
    }
    return INPUT_INVALID_NUMBER;
}
```

```
69
70  static input_error_t matrix_to_input_error(matrix_error_t error) {
71      if (error == MATRIX_OK) return INPUT_OK;
72      if (error == MATRIX_ALLOCATION_FAILED) return INPUT_ALLOCATION_FAILED;
73      return INPUT_MATRIX_SIZE_ERROR;
74  }
75
76  static input_error_t read_matrix_elements(FILE *stream, matrix_t *matrix) {
77      for (size_t row = 0; row < matrix->rows; ++row) {
78          double *values = matrix_row(matrix, row);
79          for (size_t col = 0; col < matrix->cols; ++col) {
80              token_error_t error = read_double(stream, &values[col]);
81              if (error != TOKEN_OK) return token_to_input_error(error);
82          }
83      }
84      return INPUT_OK;
85  }
86
87  input_error_t input_read(
88      FILE *stream,
89      matrix_t *source,
90      matrix_t *kernel,
91      size_t *repeat_count) {
92      if (stream == NULL || source == NULL || kernel == NULL
93          || repeat_count == NULL || source == kernel) {
94          return INPUT_INVALID_ARGUMENT;
95      }
96      if (matrix_is_valid(source) || matrix_is_valid(kernel)) {
97          return INPUT_DESTINATION_NOT_EMPTY;
98      }
99
100     size_t rows;
101     size_t cols;
102     size_t window_size;
103     size_t parsed_repeat_count;
104
105     token_error_t token_error = read_size(stream, &rows);
106     if (token_error != TOKEN_OK) return token_to_input_error(token_error);
107     token_error = read_size(stream, &cols);
108     if (token_error != TOKEN_OK) return token_to_input_error(token_error);
109     token_error = read_size(stream, &window_size);
110     if (token_error != TOKEN_OK) return token_to_input_error(token_error);
111     token_error = read_size(stream, &parsed_repeat_count);
112     if (token_error != TOKEN_OK) return token_to_input_error(token_error);
113
114     if (window_size == 0 || window_size % 2 == 0) {
115         return INPUT_INVALID_WINDOW_SIZE;
116     }
117
118     raii_stack(matrix) parsed_source = create_stack(matrix);
119     raii_stack(matrix) parsed_kernel = create_stack(matrix);
120
121     matrix_error_t matrix_error = init(matrix, parsed_source, rows, cols);
122     if (matrix_error != MATRIX_OK) return matrix_to_input_error(matrix_error);
123
124     matrix_error = init(matrix, parsed_kernel, window_size, window_size);
125     if (matrix_error != MATRIX_OK) return matrix_to_input_error(matrix_error);
126
```

```
    input_error_t input_error = read_matrix_elements(stream, &parsed_source);
    if (input_error != INPUT_OK) return input_error;

    input_error = read_matrix_elements(stream, &parsed_kernel);
    if (input_error != INPUT_OK) return input_error;

    *source = move_stack(matrix, parsed_source);
    *kernel = move_stack(matrix, parsed_kernel);
    *repeat_count = parsed_repeat_count;
    return INPUT_OK;
}

const char *input_error_string(input_error_t error) {
    switch (error) {
        case INPUT_OK:
            return "success";
        case INPUT_INVALID_ARGUMENT:
            return "invalid input_read argument";
        case INPUT_DESTINATION_NOT_EMPTY:
            return "destination matrix is not empty";
        case INPUT_UNEXPECTED_END:
            return "unexpected end of input";
        case INPUT_INVALID_NUMBER:
            return "invalid numeric value";
        case INPUT_INVALID_WINDOW_SIZE:
            return "window size must be a positive odd number";
        case INPUT_MATRIX_SIZE_ERROR:
            return "invalid or excessively large matrix size";
        case INPUT_ALLOCATION_FAILED:
            return "memory allocation failed";
        case INPUT_IO_ERROR:
            return "input/output error";
    }
    return "unknown input error";
}
```

Листинг 3: Чтение и проверка входных данных

```
#ifndef MATRIX_H
#define MATRIX_H

#include <assert.h>
#include <stddef.h>
#include <stdbool.h>
#include "common/raii.h"

typedef enum {
    MATRIX_OK = 0,
    MATRIX_INVALID_SIZE,
    MATRIX_SIZE_OVERFLOW,
    MATRIX_ALLOCATION_FAILED,
    MATRIX_SIZE_MISMATCH,
    MATRIX_INVALID_SOURCE,
    MATRIX_INVALID_DEST
} matrix_error_t;

typedef struct {
    double *data;
```

```
21      size_t rows;
22      size_t cols;
23      size_t stride;
24  } matrix_t;
25
26  void matrix_reset(matrix_t *matrix);
27  bool matrix_is_valid(const matrix_t *matrix);
28  matrix_error_t matrix_init(matrix_t *matrix, size_t rows, size_t cols);
29
30  static bool matrix_init_is_valid(matrix_error_t err) {
31      return err == MATRIX_OK;
32  }
33
34  void matrix_destroy(matrix_t *matrix);
35  void matrix_fill(matrix_t *matrix, double value);
36  matrix_error_t matrix_copy(matrix_t *destination, const matrix_t *source);
37  matrix_error_t matrix_clone(matrix_t *destination, const matrix_t *source);
38  void matrix_swap(matrix_t *left, matrix_t *right);
39
40  static matrix_error_t matrix_heap_clone(matrix_t **destination, matrix_t *const *
        source) {
41      if (unlikely(destination == NULL || *destination == NULL)) {
42          return MATRIX_ALLOCATION_FAILED;
43      }
44
45      if (unlikely(source == NULL || !matrix_is_valid(*source))) {
46          return MATRIX_INVALID_SOURCE;
47      }
48
49      return matrix_clone(*destination, *source);
50  }
51
52  static double *matrix_row(matrix_t *matrix, size_t row) {
53      assert(matrix != NULL);
54      assert(matrix_is_valid(matrix));
55      assert(row < matrix->rows);
56      return matrix->data + row * matrix->stride;
57  }
58
59  static const double *matrix_const_row(const matrix_t *matrix, size_t row) {
60      assert(matrix != NULL);
61      assert(matrix_is_valid(matrix));
62      assert(row < matrix->rows);
63      return matrix->data + row * matrix->stride;
64  }
65
66  static double *matrix_at(matrix_t *matrix, size_t row, size_t col) {
67      assert(matrix != NULL);
68      assert(matrix_is_valid(matrix));
69      assert(row < matrix->rows);
70      assert(col < matrix->cols);
71      return matrix_row(matrix, row) + col;
72  }
73
74  static const double *matrix_const_at(const matrix_t *matrix, size_t row, size_t col) {
75      assert(matrix != NULL);
76      assert(matrix_is_valid(matrix));
77      assert(row < matrix->rows);
```

```
78      assert(col < matrix->cols);
79      return matrix_const_row(matrix, row) + col;
80  }
81
82  DECLARE_SHARED_RAII_TYPES(
83      matrix, matrix_t, matrix_init, matrix_destroy, matrix_is_valid, matrix_reset)
84
85  DEFINE_RAII_TRIVIAL_MOVE(RAII_STACK_TYPE(matrix), matrix_is_valid, matrix_reset)
86  DEFINE_RAII_INIT_VALIDATION(matrix, matrix_init_is_valid)
87  DEFINE_RAII_CLONE(RAII_STACK_TYPE(matrix), matrix_clone)
88  DEFINE_RAII_CLONE(RAII_HEAP_TYPE(matrix), matrix_heap_clone)
89
90  #endif
```

Листинг 4: Интерфейс матрицы

```
1   #include "matrix.h"
2
3   #include <stdlib.h>
4   #include <stdint.h>
5   #include <string.h>
6
7   matrix_error_t matrix_init(matrix_t *matrix, size_t rows, size_t cols) {
8       if (unlikely(matrix == NULL)) return MATRIX_ALLOCATION_FAILED;
9
10      if (unlikely(matrix_is_valid(matrix))) return MATRIX_INVALID_DEST;
11
12      if (rows == 0 || cols == 0) return MATRIX_INVALID_SIZE;
13
14      if (rows > SIZE_MAX / cols) return MATRIX_SIZE_OVERFLOW;
15
16      size_t element_count = rows * cols;
17
18      if (element_count > SIZE_MAX / sizeof(double)) return MATRIX_SIZE_OVERFLOW;
19
20      double* data = (double*) malloc(element_count* sizeof(double));
21      if (unlikely(data == NULL)) return MATRIX_ALLOCATION_FAILED;
22      matrix->rows = rows;
23      matrix->cols = cols;
24      matrix->data = data;
25      matrix->stride = cols;
26      matrix_fill(matrix, 0.0);
27      return MATRIX_OK;
28  }
29
30  bool matrix_is_valid(const matrix_t *matrix) {
31      return matrix != NULL
32          && matrix->data != NULL
33          && matrix->rows > 0
34          && matrix->cols > 0
35          && matrix->stride >= matrix->cols;
36  }
37
38  void matrix_reset(matrix_t *matrix) {
39      if (unlikely(matrix == NULL)) return;
40
41      matrix->rows = 0;
42      matrix->cols = 0;
```

```c
    matrix->data = NULL;
    matrix->stride = 0;
}

void matrix_destroy(matrix_t *matrix) {
    if (unlikely(matrix == NULL)) return;

    free(matrix->data);
    matrix_reset(matrix);
}

matrix_error_t matrix_copy(matrix_t* dest, const matrix_t* src) {
    if (unlikely(!matrix_is_valid(src))) return MATRIX_INVALID_SOURCE;
    if (unlikely(!matrix_is_valid(dest))) return MATRIX_INVALID_DEST;
    if (dest == src) return MATRIX_OK;
    if (dest->rows != src->rows) return MATRIX_SIZE_MISMATCH;
    if (dest->cols != src->cols) return MATRIX_SIZE_MISMATCH;

    for (size_t i = 0; i < src->rows; ++i) {
        memcpy(matrix_row(dest, i), matrix_const_row(src, i), src->cols*sizeof(double))
            ;
    }
    return MATRIX_OK;
}

matrix_error_t matrix_clone(matrix_t *destination, const matrix_t *source) {
    if (unlikely(!matrix_is_valid(source))) return MATRIX_INVALID_SOURCE;
    if (destination == NULL || destination == source) {
        return MATRIX_INVALID_DEST;
    }

    if (unlikely(matrix_is_valid(destination))) {
        return MATRIX_INVALID_DEST;
    }

    matrix_error_t err = matrix_init(destination, source->rows, source->cols);
    if (unlikely(err != MATRIX_OK)) {
        matrix_destroy(destination);
        return err;
    }

    err = matrix_copy(destination, source);
    if (unlikely(err != MATRIX_OK)) {
        matrix_destroy(destination);
        return err;
    }
    return MATRIX_OK;
}

void matrix_swap(matrix_t *left, matrix_t *right) {
    assert(left != NULL);
    assert(right != NULL);
    if (left == right) return;

    matrix_t temp = *left;
    *left = *right;
    *right = temp;
}
```

```
100
101 void matrix_fill(matrix_t *matrix, double value) {
102     if (unlikely(!matrix_is_valid(matrix))) return;
103
104     for (size_t i = 0; i < matrix->rows; ++i) {
105         for (size_t j = 0; j < matrix->cols; ++j) {
106             *matrix_at(matrix, i, j) = value;
107         }
108     }
109 }
```

Листинг 5: Реализация матрицы

```
1  #ifndef CONVOLUTION_H
2  #define CONVOLUTION_H
3  #include "common/os/thread_pool.h"
4  #include "matrix.h"
5
6  typedef enum {
7      CONVOLUTION_OK = 0,
8      CONVOLUTION_INVALID_SOURCE,
9      CONVOLUTION_INVALID_DESTINATION,
10     CONVOLUTION_INVALID_KERNEL,
11     CONVOLUTION_SIZE_MISMATCH,
12     CONVOLUTION_IN_PLACE,
13     CONVOLUTION_ALLOCATION_FAILED,
14     CONVOLUTION_THREAD_POOL_ERROR
15 } convolution_error_t;
16
17 void convolution_apply_rows(matrix_t *destination, const matrix_t *source, const
       matrix_t *kernel,
18     size_t row_begin, size_t row_end);
19
20 convolution_error_t convolution_apply(matrix_t *destination, const matrix_t *source,
       const matrix_t *kernel);
21
22 convolution_error_t convolution_apply_repeated(matrix_t *destination, const matrix_t *
       source,
23     const matrix_t *kernel,
24     size_t repeat_count);
25
26 convolution_error_t convolution_apply_parallel(matrix_t *destination, const matrix_t *
       source,
27     const matrix_t *kernel, thread_pool_t *pool);
28
29 convolution_error_t convolution_apply_repeated_parallel(matrix_t *destination, const
       matrix_t *source,
30     const matrix_t *kernel, size_t repeat_count, thread_pool_t *pool);
31
32 const char *convolution_error_string(convolution_error_t error);
33
34 #endif //CONVOLUTION_H
```

Листинг 6: Интерфейс свёртки

```
1 #include "convolution.h"
2
```

```c
#include <stdint.h>
#include <stdlib.h>

typedef struct {
    matrix_t *destination;
    const matrix_t *source;
    const matrix_t *kernel;
    size_t row_begin;
    size_t row_end;
} convolution_task_context_t;

static const size_t convolution_min_operations_per_task = 64 * 1024;

static convolution_error_t convolution_validate_common(
    const matrix_t *destination,
    const matrix_t *source,
    const matrix_t *kernel
) {
    if (unlikely(!matrix_is_valid(destination))) return
        CONVOLUTION_INVALID_DESTINATION;
    if (unlikely(!matrix_is_valid(source))) return CONVOLUTION_INVALID_SOURCE;
    if (unlikely(!matrix_is_valid(kernel))) return CONVOLUTION_INVALID_KERNEL;

    if (unlikely(destination->rows != source->rows ||
                 destination->cols != source->cols)) {
        return CONVOLUTION_SIZE_MISMATCH;
    }

    if (unlikely(kernel->rows != kernel->cols || kernel->rows % 2 == 0)) {
        return CONVOLUTION_INVALID_KERNEL;
    }

    return CONVOLUTION_OK;
}

static convolution_error_t convolution_validate(
    const matrix_t *destination,
    const matrix_t *source,
    const matrix_t *kernel
) {
    const convolution_error_t error = convolution_validate_common(destination, source,
         kernel);
    if (unlikely(error != CONVOLUTION_OK)) return error;

    if (unlikely(destination->data == source->data || destination->data == kernel->
        data)) {
        return CONVOLUTION_IN_PLACE;
    }

    return CONVOLUTION_OK;
}

static void convolution_accumulate_row(
    double *restrict destination,
    const double *restrict source,
    size_t element_count,
    double weight
) {
```

```
58      for (size_t index = 0; index < element_count; ++index) {
59          destination[index] += source[index] * weight;
60      }
61  }
62
63  void convolution_apply_rows(
64      matrix_t *destination,
65      const matrix_t *source,
66      const matrix_t *kernel,
67      size_t row_begin,
68      size_t row_end
69  ) {
70      assert(convolution_validate(destination, source, kernel) == CONVOLUTION_OK);
71      assert(row_begin <= row_end);
72      assert(row_end <= source->rows);
73
74      const size_t radius = kernel->rows / 2;
75
76      for (size_t row = row_begin; row < row_end; ++row) {
77          double *destination_row = matrix_row(destination, row);
78          for (size_t col = 0; col < destination->cols; ++col) {
79              destination_row[col] = 0.0;
80          }
81
82          for (size_t kernel_row = 0; kernel_row < kernel->rows; ++kernel_row) {
83              size_t source_row_index;
84              if (kernel_row < radius) {
85                  const size_t offset = radius - kernel_row;
86                  if (row < offset) continue;
87                  source_row_index = row - offset;
88              } else {
89                  const size_t offset = kernel_row - radius;
90                  if (offset >= source->rows - row) continue;
91                  source_row_index = row + offset;
92              }
93
94              const double *source_row = matrix_const_row(source, source_row_index);
95              const double *reflected_kernel_row = matrix_const_row(kernel, kernel->rows
                    - 1 - kernel_row);
96
97              for (size_t kernel_col = 0; kernel_col < kernel->cols; ++kernel_col) {
98                  size_t destination_col_begin;
99                  size_t source_col_begin;
100                 size_t element_count;
101
102                 if (kernel_col < radius) {
103                     const size_t offset = radius - kernel_col;
104                     if (offset >= source->cols) continue;
105                     destination_col_begin = offset;
106                     source_col_begin = 0;
107                     element_count = source->cols - offset;
108                 } else {
109                     const size_t offset = kernel_col - radius;
110                     if (offset >= source->cols) continue;
111                     destination_col_begin = 0;
112                     source_col_begin = offset;
113                     element_count = source->cols - offset;
114                 }
```

```
115 |
116 |                 const double weight =
117 |                     reflected_kernel_row[kernel->cols - 1 - kernel_col];
118 |                 convolution_accumulate_row(
119 |                     destination_row + destination_col_begin,
120 |                     source_row + source_col_begin,
121 |                     element_count,
122 |                     weight);
123 |             }
124 |         }
125 |     }
126 | }
127 |
128 | convolution_error_t convolution_apply(matrix_t *destination, const matrix_t *source,
        const matrix_t *kernel) {
129 |     const convolution_error_t error = convolution_validate(destination, source, kernel
            );
130 |     if (unlikely(error != CONVOLUTION_OK)) return error;
131 |
132 |     convolution_apply_rows(destination, source, kernel, 0, source->rows);
133 |     return CONVOLUTION_OK;
134 | }
135 |
136 | convolution_error_t convolution_apply_repeated(matrix_t *destination, const matrix_t *
        source,
137 |     const matrix_t *kernel, size_t repeat_count) {
138 |     const convolution_error_t common_error = convolution_validate_common(destination,
            source, kernel);
139 |     if (unlikely(common_error != CONVOLUTION_OK)) return common_error;
140 |     if (unlikely(destination->data == kernel->data)) return CONVOLUTION_IN_PLACE;
141 |
142 |     if (repeat_count == 0) {
143 |         if (destination->data == source->data) return CONVOLUTION_OK;
144 |         return matrix_copy(destination, source) == MATRIX_OK
145 |             ? CONVOLUTION_OK
146 |             : CONVOLUTION_INVALID_DESTINATION;
147 |     }
148 |
149 |     if (unlikely(destination->data == source->data)) return CONVOLUTION_IN_PLACE;
150 |
151 |     raii_stack(matrix) temporary = create_stack(matrix);
152 |     if (repeat_count > 1) {
153 |         const matrix_error_t init_error = init(matrix, temporary, source->rows, source
                ->cols);
154 |         if (unlikely(init_error != MATRIX_OK)) return CONVOLUTION_ALLOCATION_FAILED;
155 |     }
156 |
157 |     const matrix_t *current_source = source;
158 |     matrix_t *current_destination = repeat_count % 2 == 0 ? &temporary : destination;
159 |
160 |     for (size_t iteration = 0; iteration < repeat_count; ++iteration) {
161 |         convolution_apply_rows(current_destination, current_source, kernel, 0, source->
                rows);
162 |         current_source = current_destination;
163 |         current_destination = current_destination == destination ? &temporary :
                destination;
164 |     }
165 |
```

```c
    return CONVOLUTION_OK;
}

static void convolution_task(void *argument) {
    convolution_task_context_t *context = argument;
    convolution_apply_rows(context->destination, context->source, context->kernel,
        context->row_begin, context->row_end);
}

static size_t convolution_task_count(
    const matrix_t *source,
    const matrix_t *kernel,
    const thread_pool_t *pool
) {
    const size_t thread_count = thread_pool_thread_count(pool);
    if (thread_count == 0) return 0;

    size_t minimum_rows_per_task = 1;
    if (kernel->rows <= convolution_min_operations_per_task / kernel->cols) {
        const size_t kernel_element_count = kernel->rows * kernel->cols;
        if (source->cols <= convolution_min_operations_per_task / kernel_element_count)
             {
            const size_t operations_per_row = source->cols * kernel_element_count;
            minimum_rows_per_task =
                convolution_min_operations_per_task / operations_per_row
                + (convolution_min_operations_per_task % operations_per_row != 0);
        }
    }

    const size_t useful_task_count = source->rows / minimum_rows_per_task
        + (source->rows % minimum_rows_per_task != 0);
    return thread_count < useful_task_count ? thread_count : useful_task_count;
}

static convolution_task_context_t *convolution_create_contexts(size_t task_count) {
    if (task_count == 0 || task_count > SIZE_MAX / sizeof(convolution_task_context_t))
         {
        return NULL;
    }
    return malloc(task_count * sizeof(convolution_task_context_t));
}

static convolution_error_t convolution_dispatch(
    matrix_t *destination,
    const matrix_t *source,
    const matrix_t *kernel,
    thread_pool_t *pool,
    convolution_task_context_t *contexts,
    thread_pool_task_t *tasks,
    size_t task_count
) {
    if (task_count == 1) {
        convolution_apply_rows(destination, source, kernel, 0, source->rows);
        return CONVOLUTION_OK;
    }

    const size_t rows_per_task = source->rows / task_count;
    const size_t remainder = source->rows % task_count;
```

```
222     size_t row_begin = 0;
223
224     for (size_t index = 0; index < task_count; ++index) {
225         const size_t row_count = rows_per_task + (index < remainder ? 1 : 0);
226         contexts[index] = (convolution_task_context_t) {
227             .destination = destination,
228             .source = source,
229             .kernel = kernel,
230             .row_begin = row_begin,
231             .row_end = row_begin + row_count
232         };
233         row_begin += row_count;
234
235         tasks[index] = (thread_pool_task_t) {
236             .func = convolution_task,
237             .arg = &contexts[index]
238         };
239     }
240
241     if (unlikely(!thread_pool_add_tasks(pool, tasks, task_count))) {
242         thread_pool_wait(pool);
243         return CONVOLUTION_THREAD_POOL_ERROR;
244     }
245     thread_pool_wait(pool);
246     return CONVOLUTION_OK;
247 }
248
249 convolution_error_t convolution_apply_parallel(matrix_t *destination, const matrix_t *
        source,
250     const matrix_t *kernel, thread_pool_t *pool) {
251     const convolution_error_t validation_error = convolution_validate(destination,
            source, kernel);
252     if (unlikely(validation_error != CONVOLUTION_OK)) return validation_error;
253     if (unlikely(thread_pool_thread_count(pool) == 0)) return
            CONVOLUTION_THREAD_POOL_ERROR;
254
255     const size_t task_count = convolution_task_count(source, kernel, pool);
256     if (task_count == 1) {
257         convolution_apply_rows(destination, source, kernel, 0, source->rows);
258         return CONVOLUTION_OK;
259     }
260
261     convolution_task_context_t *contexts = convolution_create_contexts(task_count);
262     if (unlikely(contexts == NULL)) return CONVOLUTION_ALLOCATION_FAILED;
263     thread_pool_task_t *tasks = malloc(task_count * sizeof(*tasks));
264     if (unlikely(tasks == NULL)) {
265         free(contexts);
266         return CONVOLUTION_ALLOCATION_FAILED;
267     }
268
269     const convolution_error_t error = convolution_dispatch(destination, source, kernel
            , pool,
270         contexts, tasks, task_count);
271     free(tasks);
272     free(contexts);
273     return error;
274 }
275
```

```c
convolution_error_t convolution_apply_repeated_parallel(matrix_t *destination, const matrix_t *source,
    const matrix_t *kernel, size_t repeat_count, thread_pool_t *pool) {
    const convolution_error_t common_error = convolution_validate_common(destination, source, kernel);
    if (unlikely(common_error != CONVOLUTION_OK)) return common_error;
    if (unlikely(thread_pool_thread_count(pool) == 0)) return CONVOLUTION_THREAD_POOL_ERROR;
    if (unlikely(destination->data == kernel->data)) return CONVOLUTION_IN_PLACE;

    if (repeat_count == 0) {
        if (destination->data == source->data) return CONVOLUTION_OK;
        return matrix_copy(destination, source) == MATRIX_OK
            ? CONVOLUTION_OK
            : CONVOLUTION_INVALID_DESTINATION;
    }

    if (unlikely(destination->data == source->data)) return CONVOLUTION_IN_PLACE;

    raii_stack(matrix) temporary = create_stack(matrix);
    if (repeat_count > 1) {
        const matrix_error_t init_error = init(matrix, temporary, source->rows, source->cols);
        if (unlikely(init_error != MATRIX_OK)) {
            return CONVOLUTION_ALLOCATION_FAILED;
        }
    }

    const size_t task_count = convolution_task_count(source, kernel, pool);
    convolution_task_context_t *contexts = NULL;
    thread_pool_task_t *tasks = NULL;
    if (task_count > 1) {
        contexts = convolution_create_contexts(task_count);
        if (unlikely(contexts == NULL)) return CONVOLUTION_ALLOCATION_FAILED;

        tasks = malloc(task_count * sizeof(*tasks));
        if (unlikely(tasks == NULL)) {
            free(contexts);
            return CONVOLUTION_ALLOCATION_FAILED;
        }
    }

    const matrix_t *current_source = source;
    matrix_t *current_destination = repeat_count % 2 == 0 ? &temporary : destination;
    convolution_error_t error = CONVOLUTION_OK;

    for (size_t iteration = 0; iteration < repeat_count; ++iteration) {
        error = convolution_dispatch(current_destination, current_source, kernel, pool,
            contexts, tasks, task_count);
        if (unlikely(error != CONVOLUTION_OK)) break;

        current_source = current_destination;
        current_destination = current_destination == destination ? &temporary : destination;
    }

    free(tasks);
    free(contexts);
```

```
    return error;
}

const char *convolution_error_string(convolution_error_t error) {
    switch (error) {
        case CONVOLUTION_OK:
            return "success";
        case CONVOLUTION_INVALID_SOURCE:
            return "invalid source matrix";
        case CONVOLUTION_INVALID_DESTINATION:
            return "invalid destination matrix";
        case CONVOLUTION_INVALID_KERNEL:
            return "kernel must be a valid odd square matrix";
        case CONVOLUTION_SIZE_MISMATCH:
            return "source and destination sizes do not match";
        case CONVOLUTION_IN_PLACE:
            return "in-place convolution is not supported";
        case CONVOLUTION_ALLOCATION_FAILED:
            return "convolution allocation failed";
        case CONVOLUTION_THREAD_POOL_ERROR:
            return "thread pool failed while executing convolution";
    }
    return "unknown convolution error";
}
```

Листинг 7: Последовательная и параллельная свёртка

```
#ifndef THREAD_POOL_H
#define THREAD_POOL_H
#include <stdbool.h>
#include <stddef.h>

#include "common/raii.h"

typedef void (*thread_func)(void* arg);

typedef struct {
    thread_func func;
    void* arg;
} thread_pool_task_t;

typedef enum {
    THREAD_POOL_OK = 0,
    THREAD_POOL_INVALID_SIZE,
    THREAD_POOL_INVALID_DESTINATION,
    THREAD_POOL_ALLOCATION_FAILED,
    THREAD_POOL_SYSTEM_ERROR
} thread_pool_error_t;

typedef struct thread_pool_impl_t thread_pool_impl_t;

typedef struct {
    thread_pool_impl_t *impl;
} thread_pool_t;

void thread_pool_reset(thread_pool_t *pool);
bool thread_pool_is_valid(const thread_pool_t *pool);
thread_pool_error_t thread_pool_init(thread_pool_t *pool, size_t thread_count, size_t
```

```
    queue_capacity);
bool thread_pool_add_task(thread_pool_t *pool, thread_pool_task_t task);
bool thread_pool_add_tasks(thread_pool_t *pool, const thread_pool_task_t *tasks,
    size_t task_count);
void thread_pool_wait(thread_pool_t *pool);
size_t thread_pool_thread_count(const thread_pool_t *pool);
void thread_pool_destroy(thread_pool_t *pool);
const char *thread_pool_error_string(thread_pool_error_t error);

static bool thread_pool_init_succeeded(thread_pool_error_t error) {
    return error == THREAD_POOL_OK;
}

DECLARE_SHARED_RAII_TYPES(
    thread_pool,
    thread_pool_t,
    thread_pool_init,
    thread_pool_destroy,
    thread_pool_is_valid,
    thread_pool_reset
)
DEFINE_RAII_TRIVIAL_MOVE(
    RAII_STACK_TYPE(thread_pool),
    thread_pool_is_valid,
    thread_pool_reset
)
DEFINE_RAII_INIT_VALIDATION(thread_pool, thread_pool_init_succeeded)

#endif //THREAD_POOL_H
```

Листинг 8: Интерфейс пула потоков

```
#include "thread_pool.h"

#include <pthread.h>
#include <stdlib.h>

#include "posix/mutex.h"

struct thread_pool_impl_t {
    pthread_t *threads;
    size_t thread_count;

    thread_pool_task_t *queue;
    size_t queue_capacity;
    size_t queue_head;
    size_t queue_tail;
    size_t queued_count;
    size_t active_count;

    bool stopping;
    mutex_t mutex;
    pthread_cond_t task_available;
    pthread_cond_t queue_space_available;
    pthread_cond_t idle;
};

void thread_pool_reset(thread_pool_t *pool) {
```

```
27      if (pool == NULL) return;
28      pool->impl = NULL;
29  }
30
31  bool thread_pool_is_valid(const thread_pool_t *pool) {
32      return pool != NULL && pool->impl != NULL;
33  }
34
35  static void *thread_pool_worker(void *argument) {
36      thread_pool_impl_t *pool = argument;
37
38      for (;;) {
39          thread_pool_task_t task;
40          {
41              int lock_error;
42              raii_stack(scoped_lock) _lock = new_stack(scoped_lock, &lock_error, &pool->
                    mutex);
43              if (unlikely(lock_error != 0)) return NULL;
44
45              while (pool->queued_count == 0 && !pool->stopping) {
46                  pthread_cond_wait(&pool->task_available, mutex_native(&pool->mutex));
47              }
48
49              if (pool->queued_count == 0 && pool->stopping) return NULL;
50
51              task = pool->queue[pool->queue_head];
52              pool->queue_head = (pool->queue_head + 1) % pool->queue_capacity;
53              --pool->queued_count;
54              ++pool->active_count;
55              pthread_cond_signal(&pool->queue_space_available);
56          }
57
58          task.func(task.arg);
59
60          {
61              int lock_error;
62              raii_stack(scoped_lock) _lock = new_stack(scoped_lock, &lock_error, &pool->
                    mutex);
63              if (unlikely(lock_error != 0)) return NULL;
64
65              --pool->active_count;
66              if (pool->queued_count == 0 && pool->active_count == 0) {
67                  pthread_cond_broadcast(&pool->idle);
68              }
69          }
70      }
71  }
72
73  thread_pool_error_t thread_pool_init(
74      thread_pool_t *pool,
75      size_t thread_count,
76      size_t queue_capacity
77  ) {
78      if (unlikely(pool == NULL)) return THREAD_POOL_INVALID_DESTINATION;
79      if (unlikely(thread_pool_is_valid(pool))) return THREAD_POOL_INVALID_DESTINATION;
80      if (unlikely(thread_count == 0 || queue_capacity == 0)) return
            THREAD_POOL_INVALID_SIZE;
81
```

```
82      thread_pool_impl_t *impl = calloc(1, sizeof(*impl));
83      if (unlikely(impl == NULL)) return THREAD_POOL_ALLOCATION_FAILED;
84      thread_pool_error_t error = THREAD_POOL_ALLOCATION_FAILED;
85      size_t created_thread_count = 0;
86
87      impl->threads = calloc(thread_count, sizeof(*impl->threads));
88      impl->queue = calloc(queue_capacity, sizeof(*impl->queue));
89      if (unlikely(impl->threads == NULL || impl->queue == NULL)) goto allocation_failed
            ;
90
91      impl->thread_count = thread_count;
92      impl->queue_capacity = queue_capacity;
93      mutex_reset(&impl->mutex);
94      error = THREAD_POOL_SYSTEM_ERROR;
95      if (unlikely(mutex_init(&impl->mutex) != 0)) goto allocation_failed;
96      if (unlikely(pthread_cond_init(&impl->task_available, NULL) != 0)) goto
            mutex_initialized;
97      if (unlikely(pthread_cond_init(&impl->queue_space_available, NULL) != 0)) goto
            task_condition_initialized;
98      if (unlikely(pthread_cond_init(&impl->idle, NULL) != 0)) goto
            space_condition_initialized;
99
100     for (; created_thread_count < thread_count; ++created_thread_count) {
101         if (unlikely(pthread_create(&impl->threads[created_thread_count], NULL,
102                 thread_pool_worker, impl) != 0)) {
103             goto thread_creation_failed;
104         }
105     }
106
107     pool->impl = impl;
108     return THREAD_POOL_OK;
109
110 thread_creation_failed:
111     mutex_lock(&impl->mutex);
112     impl->stopping = true;
113     pthread_cond_broadcast(&impl->task_available);
114     mutex_unlock(&impl->mutex);
115
116     for (size_t index = 0; index < created_thread_count; ++index) {
117         pthread_join(impl->threads[index], NULL);
118     }
119
120     pthread_cond_destroy(&impl->idle);
121 space_condition_initialized:
122     pthread_cond_destroy(&impl->queue_space_available);
123 task_condition_initialized:
124     pthread_cond_destroy(&impl->task_available);
125 mutex_initialized:
126     mutex_destroy(&impl->mutex);
127 allocation_failed:
128     free(impl->queue);
129     free(impl->threads);
130     free(impl);
131     return error;
132 }
133
134 bool thread_pool_add_task(thread_pool_t *pool, thread_pool_task_t task) {
135     return thread_pool_add_tasks(pool, &task, 1);
```

```
}

bool thread_pool_add_tasks(
    thread_pool_t *pool,
    const thread_pool_task_t *tasks,
    size_t task_count
) {
    if (!thread_pool_is_valid(pool)) return false;
    if (task_count == 0) return true;
    if (tasks == NULL) return false;

    thread_pool_impl_t *impl = pool->impl;
    for (size_t index = 0; index < task_count; ++index) {
        if (tasks[index].func == NULL) return false;
    }

    size_t submitted_count = 0;
    while (submitted_count < task_count) {
        int lock_error;
        raii_stack(scoped_lock) _lock = new_stack(scoped_lock, &lock_error, &impl->
            mutex);
        if (unlikely(lock_error != 0)) return false;

        while (impl->queued_count == impl->queue_capacity && !impl->stopping) {
            pthread_cond_wait(&impl->queue_space_available, mutex_native(&impl->mutex))
                ;
        }

        if (impl->stopping) return false;

        const size_t available_count = impl->queue_capacity - impl->queued_count;
        const size_t remaining_count = task_count - submitted_count;
        const size_t batch_count = available_count < remaining_count ? available_count
            : remaining_count;

        for (size_t index = 0; index < batch_count; ++index) {
            impl->queue[impl->queue_tail] = tasks[submitted_count + index];
            impl->queue_tail = (impl->queue_tail + 1) % impl->queue_capacity;
        }
        impl->queued_count += batch_count;
        submitted_count += batch_count;

        if (batch_count == 1) {
            pthread_cond_signal(&impl->task_available);
        } else {
            if (batch_count <= impl->thread_count) {
                pthread_cond_broadcast(&impl->task_available);
            } else {
                for (size_t i = 0; i < batch_count; ++i) {
                    pthread_cond_signal(&impl->task_available);
                }
            }
        }
    }

    return true;
}

```

```c
void thread_pool_wait(thread_pool_t *pool) {
    if (!thread_pool_is_valid(pool)) return;
    thread_pool_impl_t *impl = pool->impl;

    int lock_error;
    raii_stack(scoped_lock) _lock = new_stack(scoped_lock, &lock_error, &impl->mutex);
    if (unlikely(lock_error != 0)) return;

    while (impl->queued_count != 0 || impl->active_count != 0) {
        pthread_cond_wait(&impl->idle, mutex_native(&impl->mutex));
    }
}

size_t thread_pool_thread_count(const thread_pool_t *pool) {
    return thread_pool_is_valid(pool) ? pool->impl->thread_count : 0;
}

void thread_pool_destroy(thread_pool_t *pool) {
    if (!thread_pool_is_valid(pool)) return;
    thread_pool_impl_t *impl = pool->impl;

    {
        int lock_error;
        raii_stack(scoped_lock) _lock = new_stack(scoped_lock, &lock_error, &impl->
            mutex);
        if (unlikely(lock_error != 0)) return;

        impl->stopping = true;
        pthread_cond_broadcast(&impl->task_available);
        pthread_cond_broadcast(&impl->queue_space_available);
    }

    for (size_t index = 0; index < impl->thread_count; ++index) {
        pthread_join(impl->threads[index], NULL);
    }

    pthread_cond_destroy(&impl->idle);
    pthread_cond_destroy(&impl->queue_space_available);
    pthread_cond_destroy(&impl->task_available);
    mutex_destroy(&impl->mutex);
    free(impl->queue);
    free(impl->threads);
    free(impl);
    thread_pool_reset(pool);
}

const char *thread_pool_error_string(thread_pool_error_t error) {
    switch (error) {
        case THREAD_POOL_OK:
            return "success";
        case THREAD_POOL_INVALID_SIZE:
            return "thread and queue sizes must be positive";
        case THREAD_POOL_INVALID_DESTINATION:
            return "invalid thread pool destination";
        case THREAD_POOL_ALLOCATION_FAILED:
            return "thread pool allocation failed";
        case THREAD_POOL_SYSTEM_ERROR:
            return "operating system thread error";
```

```
    }
    return "unknown thread pool error";
}
```

Листинг 9: Реализация пула потоков

```
#ifndef MUTEX_H
#define MUTEX_H

#include <assert.h>
#include <errno.h>
#include <pthread.h>
#include <stdbool.h>

#include "common/raii.h"

typedef struct {
    pthread_mutex_t native;
    bool initialized;
} mutex_t;

static void mutex_reset(mutex_t *mutex) {
    if (mutex == NULL) return;
    mutex->initialized = false;
}

static bool mutex_is_valid(const mutex_t *mutex) {
    return mutex != NULL && mutex->initialized;
}

static int mutex_init(mutex_t *mutex) {
    if (mutex == NULL) return EINVAL;
    if (mutex_is_valid(mutex)) return EBUSY;

    const int error = pthread_mutex_init(&mutex->native, NULL);
    if (error == 0) mutex->initialized = true;
    return error;
}

static void mutex_destroy(mutex_t *mutex) {
    if (!mutex_is_valid(mutex)) return;

    const int error = pthread_mutex_destroy(&mutex->native);
    assert(error == 0);
    (void)error;
    mutex->initialized = false;
}

static int mutex_lock(mutex_t *mutex) {
    if (!mutex_is_valid(mutex)) return EINVAL;
    return pthread_mutex_lock(&mutex->native);
}

static int mutex_unlock(mutex_t *mutex) {
    if (!mutex_is_valid(mutex)) return EINVAL;
    return pthread_mutex_unlock(&mutex->native);
}

```

```
53 static pthread_mutex_t *mutex_native(mutex_t *mutex) {
54     return mutex_is_valid(mutex) ? &mutex->native : NULL;
55 }
56
57 static bool mutex_init_succeeded(int error) {
58     return error == 0;
59 }
60
61 DECLARE_SHARED_RAII_TYPES(
62     mutex,
63     mutex_t,
64     mutex_init,
65     mutex_destroy,
66     mutex_is_valid,
67     mutex_reset
68 )
69 DEFINE_RAII_INIT_VALIDATION(mutex, mutex_init_succeeded)
70
71 typedef struct {
72     mutex_t *mutex;
73     bool owns_lock;
74 } scoped_lock_t;
75
76 static void scoped_lock_reset(scoped_lock_t *lock) {
77     if (lock == NULL) return;
78     lock->mutex = NULL;
79     lock->owns_lock = false;
80 }
81
82 static bool scoped_lock_is_valid(const scoped_lock_t *lock) {
83     return lock != NULL && lock->owns_lock && mutex_is_valid(lock->mutex);
84 }
85
86 static int scoped_lock_init(scoped_lock_t *lock, mutex_t *mutex) {
87     if (lock == NULL || !mutex_is_valid(mutex)) return EINVAL;
88     if (scoped_lock_is_valid(lock)) return EBUSY;
89
90     const int error = mutex_lock(mutex);
91     if (error == 0) {
92         lock->mutex = mutex;
93         lock->owns_lock = true;
94     }
95     return error;
96 }
97
98 static void scoped_lock_destroy(scoped_lock_t *lock) {
99     if (!scoped_lock_is_valid(lock)) return;
100
101     const int error = mutex_unlock(lock->mutex);
102     assert(error == 0);
103     (void)error;
104     scoped_lock_reset(lock);
105 }
106
107 static bool scoped_lock_init_succeeded(int error) {
108     return error == 0;
109 }
110
```

```
DECLARE_SHARED_RAII_TYPES(
    scoped_lock,
    scoped_lock_t,
    scoped_lock_init,
    scoped_lock_destroy,
    scoped_lock_is_valid,
    scoped_lock_reset
)
DEFINE_RAII_TRIVIAL_MOVE(
    RAII_STACK_TYPE(scoped_lock),
    scoped_lock_is_valid,
    scoped_lock_reset
)
DEFINE_RAII_INIT_VALIDATION(scoped_lock, scoped_lock_init_succeeded)

#endif //MUTEX_H
```

Листинг 10: POSIX-мьютекс и scoped lock

```
#ifndef RAII_H
#define RAII_H

#include <stdbool.h>
#include <stdlib.h>
#include "unlikely.h"

#if defined(__clang__) || defined(__GNUC__)

#define RAII_DETAIL_CAT_IMPL(left, right) left##right
#define RAII_DETAIL_CAT(left, right) RAII_DETAIL_CAT_IMPL(left, right)
#define RAII_DETAIL_SYMBOL(TypeName, suffix) \
    RAII_DETAIL_CAT(RAII_DETAIL_CAT(raii_detail_, TypeName), suffix)
#define RAII_STACK_TYPE(TypeName) RAII_DETAIL_CAT(stack_, TypeName)
#define RAII_HEAP_TYPE(TypeName) RAII_DETAIL_CAT(heap_, TypeName)

/*
 * ResetFunc must be infallible, must not inspect the previous value and must
 * leave the object in a canonical invalid state. CleanupFunc is called only
 * for a value accepted by IsValidFunc.
 */
#define DEFINE_RAII_TYPE(TypeName, BaseType, CleanupFunc, IsValidFunc, ResetFunc) \
    typedef BaseType TypeName;\
    static inline void RAII_DETAIL_SYMBOL(TypeName, _cleanup)(TypeName *ptr) { \
        if (unlikely(!ptr)) return; \
        if (likely(IsValidFunc(ptr))) { \
            CleanupFunc(ptr); \
            ResetFunc(ptr); \
        } \
    }

/* Use only when TypeName can be safely moved with ordinary assignment. */
#define DEFINE_RAII_TRIVIAL_MOVE(TypeName, IsValidFunc, ResetFunc) \
    static inline TypeName RAII_DETAIL_SYMBOL(TypeName, _take)(TypeName *source) { \
        TypeName val = {0}; \
        ResetFunc(&val); \
        if (likely(source && IsValidFunc(source))) { \
            val = *source; \
            ResetFunc(source); \
```

```
40          } \
41          return val; \
42      } \
43      static inline void RAII_DETAIL_SYMBOL(TypeName, _move_assign)( \
44              TypeName *destination, TypeName *source) { \
45          if (unlikely(!destination || destination == source)) return; \
46          RAII_DETAIL_SYMBOL(TypeName, _cleanup)(destination); \
47          if (likely(source && IsValidFunc(source))) { \
48              *destination = *source; \
49              ResetFunc(source); \
50          } else { \
51              ResetFunc(destination); \
52          } \
53      }
54
55  /*
56   * CloneFunc must have the form:
57   * ErrorType clone(TypeName *destination, const TypeName *source)
58   * destination must be distinct and contain no live resource (normally it is
59   * produced by create()). On failure it must remain without a live resource,
60   * so its automatic cleanup is always safe.
61   */
62  #define DEFINE_RAII_CLONE(TypeName, CloneFunc) \
63      static inline __typeof__(CloneFunc) * \
64      RAII_DETAIL_SYMBOL(TypeName, _get_clone)(void) { \
65          return CloneFunc; \
66      }
67
68  #define DEFINE_RAII_INIT(BaseName, InitFunc) \
69      static inline __typeof__(InitFunc) *RAII_DETAIL_SYMBOL(BaseName, _get_init)(void)
            { \
70          return InitFunc; \
71      } \
72
73  #define DEFINE_RAII_INIT_VALIDATION(BaseName, IsInitSuccessFunc) \
74      static inline __typeof__(IsInitSuccessFunc) *RAII_DETAIL_SYMBOL(BaseName,
            _get_init_valid)(void) { \
75          return IsInitSuccessFunc; \
76      } \
77
78  #define init_succeeded(BaseName, result) (RAII_DETAIL_SYMBOL(BaseName, _get_init_valid
        )())(result)
79
80  #define init_failed(BaseName, result) !init_succeeded(BaseName, result)
81
82  #define DECLARE_SHARED_RAII_TYPES(BaseName, BaseType, \
83      InitFunc, DestroyFunc, ValidFunc, ResetFunc) \
84      DEFINE_RAII_TYPE(RAII_STACK_TYPE(BaseName), BaseType, DestroyFunc, ValidFunc,
            ResetFunc) \
85      \
86      DEFINE_RAII_INIT(BaseName, InitFunc) \
87      static inline void RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _destroy)(BaseType
             **ptr) { \
88          if (unlikely(!ptr || !*ptr)) return; \
89          if (ValidFunc(*ptr)) { \
90              DestroyFunc(*ptr); \
91          } \
92          free(*ptr); \
```

```
93          *ptr = NULL; \
94      } \
95      static inline void RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _reset)(BaseType
            **ptr) { \
96          if (unlikely(!ptr)) return; \
97          *ptr = NULL; \
98      } \
99      static inline bool RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _valid)(BaseType
            **ptr) { \
100         return ptr != NULL && *ptr != NULL; \
101     } \
102     DEFINE_RAII_TYPE(RAII_HEAP_TYPE(BaseName), BaseType*, \
103                     RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _destroy), \
104                     RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _valid), \
105                     RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _reset)) \
106     DEFINE_RAII_TRIVIAL_MOVE( \
107         RAII_HEAP_TYPE(BaseName), \
108         RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _valid), \
109         RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _reset)) \
110     \
111     static inline RAII_STACK_TYPE(BaseName) \
112     RAII_DETAIL_SYMBOL(RAII_STACK_TYPE(BaseName), _create_empty)(void) { \
113         BaseType var = {0}; \
114         ResetFunc(&var); \
115         return var; \
116     } \
117     \
118     static inline RAII_HEAP_TYPE(BaseName) \
119     RAII_DETAIL_SYMBOL(RAII_HEAP_TYPE(BaseName), _create_empty)(void) { \
120         BaseType *var = malloc(sizeof(*var)); \
121         if (likely(var)) { \
122             ResetFunc(var); \
123         } \
124         return var; \
125     }
126
127 #define raii(TypeName) \
128     TypeName __attribute__((cleanup(RAII_DETAIL_SYMBOL(TypeName, _cleanup))))
129
130 #define raii_stack(TypeName) raii(RAII_STACK_TYPE(TypeName))
131 #define raii_heap(TypeName) raii(RAII_HEAP_TYPE(TypeName))
132
133 #define create(TypeName) RAII_DETAIL_SYMBOL(TypeName, _create_empty)()
134 #define create_stack(BaseName) create(RAII_STACK_TYPE(BaseName))
135 #define create_heap(BaseName) create(RAII_HEAP_TYPE(BaseName))
136
137 #define init(BaseName, var, ...) \
138     (RAII_DETAIL_SYMBOL(BaseName, _get_init)())( \
139         _Generic(&(var), \
140             RAII_STACK_TYPE(BaseName)*: &(var), \
141             RAII_HEAP_TYPE(BaseName)*: (var) \
142         ), \
143         ##__VA_ARGS__ \
144     )
145
146 #define drop(TypeName, var) RAII_DETAIL_SYMBOL(TypeName, _cleanup)(&(var))
147 #define drop_stack(BaseName, var) drop(RAII_STACK_TYPE(BaseName), var)
148 #define drop_heap(BaseName, var) drop(RAII_HEAP_TYPE(BaseName), var)
```

```
149 |
150 | #define move(TypeName, var) RAII_DETAIL_SYMBOL(TypeName, _take)(&(var))
151 | #define move_stack(BaseName, var) move(RAII_STACK_TYPE(BaseName), var)
152 | #define move_heap(BaseName, var) move(RAII_HEAP_TYPE(BaseName), var)
153 |
154 | /* ONLY FOR TRIVIAL MOVE. DO NOT USE WITH NON-TRIVIAL OR NON-MOVEABLE OBJECTS(e.g.
      Mutex) */
155 | #define new_stack(BaseName, init_result_ptr, ...) \
156 | ({ \
157 |     RAII_STACK_TYPE(BaseName) raii_new_value = create_stack(BaseName); \
158 |     __auto_type raii_new_result = \
159 |     init(BaseName, raii_new_value, ##__VA_ARGS__); \
160 |     *(init_result_ptr) = raii_new_result; \
161 |     if (init_failed(BaseName, raii_new_result)) { \
162 |         drop_stack(BaseName, raii_new_value); \
163 |     } \
164 |     move_stack(BaseName, raii_new_value); \
165 | })
166 |
167 | #define new_heap(BaseName, init_result_ptr, ...) \
168 | ({ \
169 |     RAII_HEAP_TYPE(BaseName) raii_new_value = create_heap(BaseName); \
170 |     __auto_type raii_new_result = \
171 |     init(BaseName, raii_new_value, ##__VA_ARGS__); \
172 |     *(init_result_ptr) = raii_new_result; \
173 |     if (init_failed(BaseName, raii_new_result)) { \
174 |     drop_heap(BaseName, raii_new_value); \
175 |     } \
176 |     move_heap(BaseName, raii_new_value); \
177 | })
178 |
179 | #define move_assign(TypeName, destination, source) \
180 |     RAII_DETAIL_SYMBOL(TypeName, _move_assign)(&(destination), &(source))
181 | #define move_assign_stack(BaseName, destination, source) \
182 |     move_assign(RAII_STACK_TYPE(BaseName), destination, source)
183 | #define move_assign_heap(BaseName, destination, source) \
184 |     move_assign(RAII_HEAP_TYPE(BaseName), destination, source)
185 |
186 |
187 | #define try_clone(TypeName, destination, source) \
188 |     (RAII_DETAIL_SYMBOL(TypeName, _get_clone)())(&(destination), &(source))
189 | #define try_clone_stack(BaseName, destination, source) \
190 |     try_clone(RAII_STACK_TYPE(BaseName), destination, source)
191 | #define try_clone_heap(BaseName, destination, source) \
192 |     try_clone(RAII_HEAP_TYPE(BaseName), destination, source)
193 |
194 | #else
195 | #error "This project requires GCC or Clang for RAII support"
196 | #endif
197 |
198 | #endif //RAII_H
```

Листинг 11: RAII-подобное управление ресурсами

```
1 | #ifndef UNLIKELY_H
2 | #define UNLIKELY_H
3 | #if defined(__GNUC__) || defined(__clang__)
4 | #define unlikely(x) __builtin_expect(!!(x), 0)
```

```
#define likely(x) __builtin_expect(!!(x), 1)
#else

#define unlikely(x) (x)
#define likely(x) (x)
#endif

#endif //UNLIKELY_H
```

Листинг 12: Подсказки предсказателю переходов

```
cmake_minimum_required(VERSION 3.30)
project(os_lab3 C)

set(CMAKE_C_STANDARD 11)

set(THREADS_PREFER_PTHREAD_FLAG ON)
find_package(Threads REQUIRED)

add_executable(os_lab3 main.c
        common/os/thread_pool.h
        common/os/thread_pool.c
        common/raii.h
        common/unlikely.h
        common/os/posix/threads.h
        common/std.h
        common/os/posix/mutex.h
        matrix.h
        matrix.c
        input.h
        input.c
        convolution.c
        convolution.h)

target_include_directories(os_lab3 PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}
)

target_link_libraries(os_lab3 PRIVATE Threads::Threads)
```

Листинг 13: Конфигурация CMake